

CISCO CERTIFIED CYBEROPS ASSOCIATE (CBROPS 200-201)

Lab 17.2.7: Reading Server Logs

Log Analysis Utilities (cat, more, less, tail -f), Syslog Rotation, and Systemd Journalctl Operations

COURSE MODULE	LOG ARCHITECTURES	UTILITIES MASTERED	WORKSTATION ENVIRONMENT
Module 17: Attacking What We Do	Plain-Text Syslog & Binary Journald	cat, more, less, tail -f, journalctl	CyberOps Workstation VM (secOps)

Objectives

- **Part 1: Reading Log Files with Cat, More, Less, and Tail** — Master CLI text utilities, compare navigation capabilities, and actively follow live log streams.
- **Part 2: Log Files and Syslog Architecture** — Investigate centralized syslog formatting, analyze root permission requirements, and inspect log rotation mechanics.
- **Part 3: Log Files and Journalctl Operations** — Query systemd binary journals, filter by boot ID, timestamp/UTC, kernel facility, and specific systemd services.
- **Comparative Evaluation & SIEM Integration** — Rigorously compare Syslog vs. Systemd Journald advantages, limitations, and forensic considerations.

Background / Scenario

Server and application logs are the primary telemetry source for security analysts detecting intrusions, diagnosing system failures, and reconstructing incident timelines. Operating systems generate distinct log structures — ranging from legacy ASCII plain-text files managed by Syslog daemons to structured binary logs managed by systemd's `journald`.

A CyberOps analyst must be proficient with foundational Linux utilities (`cat` , `more` , `less` , `tail`) and advanced binary log management tools (`journalctl`) to rapidly extract actionable security intelligence during triage and active investigations.

Part 1: Reading Log Files with Cat, More, Less, and Tail

Step 1: Comparing Static File Viewers (cat, more, less)

Analysts examine an Apache access log file (`logstash-tutorial.log`) using three primary text viewers:

- `cat /home/analyst/lab.support.files/logstash-tutorial.log` — Concatenates and dumps the entire file to stdout without paging. Fast for small files, but overflows terminal buffers for enterprise logs.
- `more /home/analyst/lab.support.files/logstash-tutorial.log` — Paginates file output, advancing one screen per Spacebar and one line per Enter.
- `less /home/analyst/lab.support.files/logstash-tutorial.log` — Advanced pager supporting bidirectional navigation, text searching, and memory-efficient chunk loading.

STEP 1.C — CLI TOOL EVALUATION

What is the drawback of using `more`?

ANALYST TECHNICAL EVALUATION:

The primary drawback of `more` is that it is **strictly unidirectional (forward-only)**. Users can only advance downward through the file; they cannot scroll backward (upward) to review previously displayed lines without exiting and restarting the command.

Advantages of `less`: In contrast, `less` ("less is more") provides full bidirectional scrolling (↑/↓, Page Up/Page Down), forward searching (`/pattern`), backward searching (`?pattern`), and does not require loading the entire file into RAM before viewing.

Step 2: Actively Following Live Logs with `tail -f`

In security operations, analysts must observe logs in real time as events occur. Running `tail -f` continuously monitors a file descriptor for new append operations:

```
# Terminal 1 (Live Monitoring):
analyst@secOps ~$ sudo tail -f /home/analyst/lab.support.files/logstash-tutorial.log

# Terminal 2 (Simulating Log Ingestion):
analyst@secOps ~$ echo "this is a new entry to the monitored log file" >> lab.support.files/logstash-tutorial.log
```

Terminal 1 instantly prints the appended string to the console without delay, demonstrating the real-time event streaming capability required for live intrusion monitoring.

Part 2: Log Files and Syslog Architecture

Syslog standardizes system and application event logging. The CyberOps VM records operating system events to `/var/log/syslog`.

PART 2.A — SECURITY PERMISSIONS & ACCESS CONTROL

Why did the `cat` command have to be executed as `root`?

SOC SECURITY POLICY FINDING:

System log files located in `/var/log/` contain sensitive security telemetry, including system authentication attempts, user activities, daemon errors, IP connections, and kernel diagnostics.

To prevent unauthorized information disclosure and preserve confidentiality, POSIX permissions on `/var/log/syslog` are restricted (typically `-rw-r-----` or `0640` owned by `root:adm`). Standard non-privileged user accounts (such as `analyst`) lack read permissions and must elevate via `sudo`.

PART 2.B — NETWORK TIME PROTOCOL (NTP) & LOG SYNCHRONIZATION

Can you think of a reason why it is so important to keep the time and date of computers correctly synchronized?

SOC FORENSIC & INCIDENT RESPONSE RATIONALE:

- **Multi-Source Incident Reconstruction:** Correlating security events across distributed firewalls, domain controllers, cloud workloads, and SIEM platforms requires microsecond-accurate, synchronized timestamps. Inaccurate time skews the attack narrative, making incident reconstruction impossible.
- **Authentication Protocol Validity:** Cryptographic authentication mechanisms (Kerberos ticket granting, OAuth2/SAML assertion tokens, Time-based One-Time Passwords [TOTP]) fail if endpoint clocks drift beyond tight skew thresholds (typically ± 5 minutes for Kerberos).
- **Certificate & Key Lifecycle:** Validating X.509 TLS/SSL certificates, CRLs (Certificate Revocation Lists), and OCSP stapling relies on precise system time to enforce validity periods.
- **Legal Admissibility:** Digital evidence presented in judicial proceedings or regulatory compliance audits (PCI-DSS, ISO 27001, HIPAA) requires a verifiable, synchronized chain of custody.

Part 3: Log Files and Systemd Journalctl Operations

Systemd's `journal` service stores system telemetry in an optimized, append-only binary format. The `journalctl` utility provides rich filtering capabilities:

COMMAND	PRIMARY FUNCTION	SOC ANALYTICAL USE CASE
<code>journalctl</code>	Displays all recorded binary logs from beginning.	Complete chronological system audit.
<code>sudo journalctl --utc</code>	Converts all displayed timestamps to UTC.	Standardizing timeline against external SIEM logs.
<code>sudo journalctl -b</code>	Filters logs recorded only during the current boot.	Diagnosing current session crashes and startups.
<code>sudo journalctl -u nginx.service --since today</code>	Filters by systemd unit (<code>nginx</code>) starting at 00:00:00 today.	Investigating web application attacks occurring today.
<code>sudo journalctl -k</code>	Displays only Linux kernel messages (equivalent to <code>dmesg</code>).	Hardware errors, kernel panics, driver issues.
<code>sudo journalctl -f</code>	Actively follows the live journal log stream in real time.	Live threat hunting and real-time event monitoring.

Reflection & Comparative Architecture Analysis

REFLECTION QUESTION — SYSLOG VS. SYSTEMD JOURNALD EVALUATION

Compare Syslog and Journald. What are the advantages and disadvantages of each?

COMPARATIVE TECHNICAL BREAKDOWN:

DIMENSION	SYSLOG ARCHITECTURE (RSYSLOG / SYSLOG-NG)	SYSTEMD JOURNALD ARCHITECTURE (JOURNALCTL)
Storage Format	Human-readable ASCII plain-text files (<code>/var/log/</code>).	Structured, indexed append-only binary files (<code>/var/log/journal/</code>).
Metadata & Indexing	Basic text: Timestamp, Hostname, Process[PID], Message.	Rich structured key-value metadata: PID, UID, GID, Unit, Boot ID, SELinux context.
Tooling & Querying	Standard text tools (<code>grep</code> , <code>awk</code> , <code>sed</code> , <code>cat</code> , <code>less</code>).	Requires <code>journalctl</code> utility; allows fast indexed filtering by field/service/time.
Log Rotation	Requires external service (<code>logrotate</code> cron job).	Built-in automatic size cap and retention rotation management.
Tamper Resistance	Low — Plain-text files can be modified if root is compromised.	High — Built-in Forward Secure Sealing (FSS) cryptographic verification.
Network Centralization	Native, ubiquitous protocol supported by virtually all network appliances.	Local host focus; requires forwarding daemon (<code>systemd-journal-remote</code> or <code>rsyslog</code>).
Advantages	Universal compatibility, plain-text simplicity, easy network forwarding.	Rich metadata, high query performance, early boot capture, tamper sealing.
Disadvantages	Unstructured text requires custom regex parsing; no binary integrity check.	Binary corruption can destroy logs; not portable across non-systemd OSes.

Enterprise Logging Best Practice Architecture

- Hybrid Architecture** Utilize **systemd-journald** for high-fidelity local endpoint logging, forwarding structured events via **rsyslog / syslog-ng** to an enterprise SIEM (Splunk, Elastic Security, Microsoft Sentinel).
- Cryptographic Sealing** Enable **Forward Secure Sealing (FSS)** in journald (`journalctl --setup-keys`) to detect any post-incident log tampering by rootkits.
- NTP Synchronization** Enforce network-wide chrony/NTP synchronization across all nodes to guarantee sub-millisecond timestamp alignment.
- WORM Storage** Ingest centralized logs into Write-Once-Read-Many (WORM) cloud object storage or immutable log servers to satisfy compliance and evidentiary standards.